Contents

Project Description: 2
Scope: 2
Project Requirements: 3
Database Requirements: 3
User Requirements: 4
Business Rules: 4
Entity Relationship Diagram (ERD): 5
Data Dictionary: 6
Basic Concepts of the Data Dictionary: 7
1) Entity Generation and Data Entry for Table Claim: 9
2) Entity Generation and Data Entry for Table Policy: 11
3) Entity Generation and Data Entry for Table PolicyCoverage: 13
4) Entity Generation and Data Entry for Table CoveredPeople: 15
5) Entity Generation and Data Entry for Table VehicleDetails: 16
6) Entity Generation and Data Entry for Table History: 18
7) Entity Generation and Data Entry for Table Employer: 20
8) Entity Generation and Data Entry for Table Exposure: 22
9) Entity Generation and Data Entry for Table FNOL: 23
10) Entity Generation and Data Entry for Table Financial: 25
11) Entity Generation and Data Entry for Table LossType: 27
Data Retrieval and Simple Reports: 29
Data Analysis: 29
Analysis -1 : 29
Analysis – 2: 30
Analysis -3: 31
Analysis 4: 32
Analysis – 5: 33
Analysis – 6: 34
Analysis – 7: 35
Analysis – 8: 36
Analysis-9: 37
Analysis – 10: 39

Project Description:

This Insurance claim management system is a sophisticated and comprehensive solution aimed at optimizing the way insurance companies manage their claims. It is designed while keeping in mind the scalability factor as well as maintaining efficiency for larger inventories. Claim management system addresses critical problems that insurance providers face in their day-to-day work. By utilizing modern database technologies, the management system effortlessly integrates the claim data with the transactional data and policy details.

A unified database containing all required data at one place would drastically improve the efficiency of the database while maintaining integrity. This project uses that method to eliminate any data silos and make sure that the data being worked upon is consistent with accurate data. From the initial phase of the claim which is the First notice of the loss (FNOL) to the end final settlement of the loss, this CMS provides a robust framework to analyze and manage the claims.

Objective:

The Claim Management System represents a sophisticated and all-encompassing solution aimed at transforming the methods employed by insurance firms in managing claims. Through the incorporation of processes for claim initiation, monitoring, and resolution, the system intends to eradicate inefficiencies while improving operational precision. It enables customer service representatives to document claim information with accuracy, thereby lessening errors and diminishing processing duration. Automation and integrated data consolidate client satisfaction, faster claims settlement, and allow transparency in all respects. The system integrates various aspects relating to the claims management process for policy information, financial transactions involved, and loss-related information so as to provide a single-window framework enabling efficiency as well as consistency.

It also enables teamwork within the personnel in an easy-to-use interface, with real-time data access and controlled function-based access, thereby enhancing the overall processing efficiency and reliability. Precisely, it is targeted at ensuring the delivery of high-quality service, easing administrative tasks, and ensuring the growth of any concerned organization.

Scope:

The Claim Management System aims at changing the trend in the insurance claim process, handling all aspects of claim management right from origination to settlement. The system will allow the creation of unique claim numbers assigned for every incident reported, usually referred to as First Notice of Loss-FNOL. This would be a very important identifier in relationship to a policy table that holds everything about the customer's policy and coverage. Each documented loss

gets classified by certain placeholders, known as exposures, that define the coverage type applied on the loss. These exposures then link with the financials table, which captures the payment information to allow for correct and timely reimbursements. This system also provides communication between the customer and the employees. If a loss occurs, the customer calls a service representative. The service representative builds the claim details in the system. The system keeps track of everything the representative does to create, modify, or remove records of the actions to the claim history table so that there is accountability and auditing. The system shall also have role-based access-for example, only relevant persons shall have access to confidential information.

Besides, the Claims Management System makes the process of insurance smooth by allowing real-time status tracking, detailed reporting, and automation of workflow processes. It improves coordination between various departments due to the integration of policy information, claim documents, and financial records on a single platform. It also helps in efficient resource allocation by giving the necessary insight into claim trends and other payment patterns. In enabling faster and more accurate processing, therefore, the system satisfies regulatory requirements by enhancing the totality of customer experiences and positions insurance companies to customers as reliable yet prompt responders to customer needs.

Project Requirements:

- **Operating System:** Windows
- **Database:** MySQL
- **Applications:** MYSQL Workbench, draw.io, Microsoft Word, Microsoft Excel

Database Requirements:

The following is the list of the various data tables that will be used.

- **Claim Table**
- **Policy Table**
- **PolicyCoverage Table**
- **CoveredPeople Table**
- **VehicleDetails Table**
- **History Table**
- **Employer Table**
- **Exposure Table**
- **FNOL Table**
- **Financial Table**
- **LossType Table**
- **LossState Table**

User Requirements:

1) **Claim Registration:** To be able to open up a new claim in the customer service representative, which automatically flags it as the First Notice of Loss-unique number-identified for each claim.
2) **FNOL Information:** Comprehensive details, encompassing the date of loss and the corresponding policy number, must be submitted when initiating a claim.
3) **Vehicle Damages and Casualties Information:** Vehicle damage claims require recording at least one injury detail or one vehicle damage detail.
4) **Making Exposures:** The exposures should be calculated based on the amount of insurance carried when a policy was underwritten.
5) **Editing Exposure:** Agents should be able to fill in the system the following underwriting exposure information: amount claimed, coverage type, and details of loss.
6) **Payment Logging:** All transactions, including all changes, need to be tracked in the system with no loss of data, but also with a connection to related claims.
7) **Payment Constraining:** The compensation can be made only for the exposure created, while the amount compensated must be less than or equal to the compensation amount.
8) **Insurance Coverage Verification:** Agents should be allowed to add insurance directly from the policy while creating exposures.
9) **History Event Limitations:** The users cannot change all the past events recorded in the system because they are kept for future reference.
10) **Active policy requirement:** A claim can only be filed when the loss arises under an active policy.
11) **Claim Summary and Status:** Agents must be given access to summaries of claims and exposure regarding the status that could be open, closed, and pending payments.

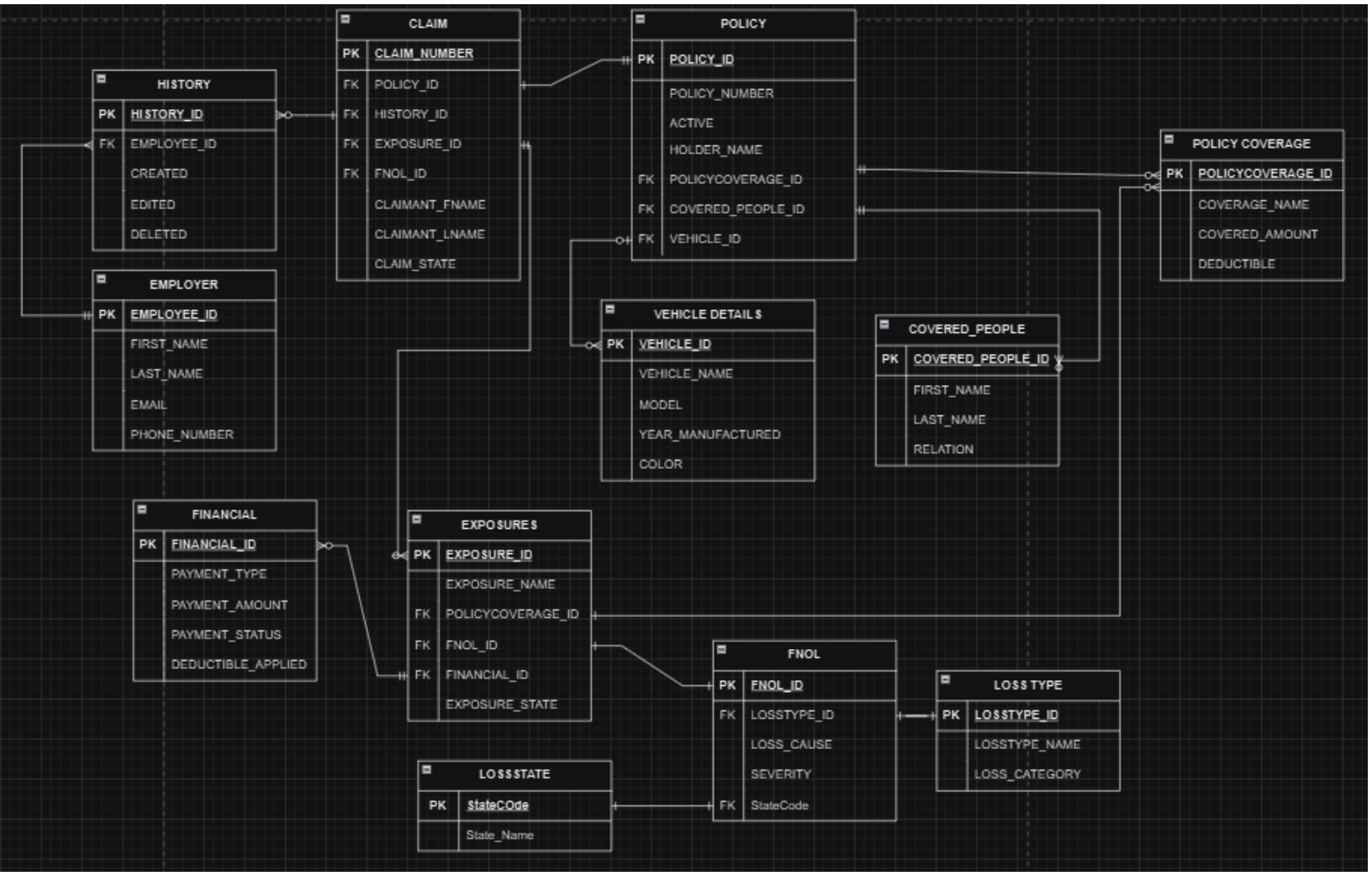
Business Rules:

1) **Identifying Distinctive Claims:** A different number identifies each claim for efficient tracking and management.
2) **Compulsory Claim Information:** It requires basic information about the policy number, claim amount, and incident date to file the claim.
3) **Policy Coverage Verification:** First, each claim must be verified to determine if the claim is within policy coverage.
4) **Claims Status Tracking:** The system will show different claim statuses at different stages, like "Pending," "In Review," "Approved," or "Rejected," depending on the processing stage.
5) **Role-Based Access Control:** Access to view or edit a claim is based on the user's role, be it a customer service representative or a manager.
6) **Auditing Logging:** The system keeps a record of all edits to the claims information, including the editor and time of edit.

7) **Duplicate Claims Avoidance:** Mechanisms are in place to avoid duplicate claims filing for the same incidence.
8) **Document Storage and Linking:** The relevant documents for incident reports and medical treatments are kept on secured files coupled with the case.
9) **Regulatory Compliance:** The system follows the data protection policy, hence the safety and privacy of customer information.
10) **Payment Authorization Control:** Payments require approval from designated staff members, ensuring they align with policy details before disbursement.

Entity Relationship Diagram (ERD):

The following diagram represents the entity relationship in the database.



The ERD for the Claim Management System includes those 12 entities, capturing various aspects of the claim process, including but not limited to claims, policies, exposures, financial transactions, history, and FNOL-first notice of loss-which would be involved in providing functional claim tracking and processing within the insurance context.

Components of the ERM:

TABLES	TABLE RELATIONSHIPS
Employer ↔ History	One to Many (1:M)
Claim ↔ History	One to Many (1:M)
Claim ↔ Policy	One to One (1:1)
Claim ↔ Exposure	One to Many (1:M)
Policy ↔ Policy Coverage	One to Many (1:M)
Policy ↔Vehicle Details	One to Many (1:M)
Policy ↔ Covered People	One to Many (1:M)
Exposure ↔ Policy Coverage	One to Many (1:M)
Exposure ↔ Financial	One to Many (1:M)
Exposure ↔ FNOL	One to One (1:1)
FNOL ↔ Loss State	One to One (1:1)
FNOL ↔ Loss Type	One to One (1:1)

Data Dictionary:

The data dictionary plays a vital role in the design of the database: it acts as a central store whereby information related to data elements within a system is stored. It enhances uniformity, clarity, and standardization of all data assets by defining important attributes like data names, types, formats, and ranges. The mapping of tables and their relationships enhances traceability with the data dictionary and helps in the interaction whereby good communication among team members is established.

Basic Concepts of the Data Dictionary:

- Ensures there is no duplication of data within the warehouse data management system to avoid errors and inconsistencies.
- Performs complete documentation of naming convention, data formats, and their usage within the system for consistency in interpretation.
- Aligns team members regarding data collection, management, and interpretation for effective collaboration and avoidance of misunderstandings.
- Enforces standard rules in creating, changing, and utilizing data, making sure quality and integrity are maintained in the system.
- It can enhance the analysis process in defined definitions and relationships, thus aiding in the quicker discovery of business insights.

(Note: Please double-click on the below file to view the data dictionary.)

CLAIM MANAGEMENT SYSTEM

TABLE NAME	Field/Display name	Description	Data Type	Field Length	Data Format	Required	Nullability	PK/FK	Validation Rules	Example
EMPLOYER	Employee ID	Employer Identifier	VARCHAR	10	N/A	Yes	No	PK	Unique	18ICP104
	First Name	First Name of the Employee	VARCHAR	50	Text	Yes	No	-	Text only	Shivani
	Last Name	Last Name of the Employee	VARCHAR	50	Text	Yes	No	-	Text only	Raghu
	Email	Email Address	VARCHAR	100	Email	Yes	No	-	Email format	s_raghu@gmail.com
	Phone Number	Contact Number	VARCHAR	15	Phone	Yes	No	-	Phone format	123-456-7890
HISTORY	History ID	History Record Identifier	INT	10	N/A	Yes	No	PK	Unique	999999
	Employee ID	Reference to Employee	VARCHAR	10	N/A	Yes	No	FK	Must exist in Employee table	18ICP104
	Created	Record Creation Date	DATETIME	N/A	MM/DD/YYYY HH:MM:SS	Yes	No	-	Date format	1/6/2022 2:32:10 PM
	Edited	Record Edited Date	DATETIME	N/A	MM/DD/YYYY HH:MM:SS	No	Yes	-	Date format	8/6/2023 2:32:10 PM
	Deleted	Record Deleted Date	DATETIME	N/A	MM/DD/YYYY HH:MM:SS	No	Yes	-	Date format	12/7/2024 2:32:10 PM
CLAIM	Claim ID	Claim Identifier	VARCHAR	10	N/A	Yes	No	PK	Unique	CLAIM-9999
	Policy ID	Related Policy Identifier	VARCHAR	10	N/A	Yes	No	FK	Must exist in Policy table	POLID-9999
	FNOL ID	First Notice of Event Identifier	VARCHAR	10	N/A	Yes	No	FK	Must exist in FNOL table	FNOL-9999
	Claimant First Name	First Name of the Claimant	VARCHAR	50	Text	Yes	No	-	Text only	Sowmya
	Claimant Last Name	Last Name of the Claimant	VARCHAR	50	Text	Yes	No	-	Text only	Achar
	Exposure ID	Related Exposure Identifier	VARCHAR	10	N/A	Yes	No	FK	Must exist in Exposure table	EXPID-9999
	History ID	History Record Identifier	INT	10	N/A	Yes	No	PK	Unique	999999
	Claim State	Claim is applied or not	BOOLEAN	N/A	N/A	Yes	No	-	Active/Inactive	Active
EXPOSURE	Exposure ID	Exposure Identifier	VARCHAR	10	N/A	Yes	No	PK	Unique	EXPID-9999
	Exposure Name	Name of Exposure	VARCHAR	100	Text	Yes	No	-	Text only	Fire Damage
	Policy ID	Related Policy Identifier	VARCHAR	10	N/A	Yes	No	FK	Must exist in Policy table	PCovID-9999
	FNOL ID	First Notice of Event ID	VARCHAR	10	N/A	Yes	No	FK	Must exist in FNOL table	FNOL-9999
	Financial ID	Related Financial ID	VARCHAR	10	N/A	Yes	No	FK	Must exist in Financial table	FinID-9999
	Exposure State	State of the exposure	BOOLEAN	N/A	N/A	Yes	No		Open/Close	Close
FNOL (FIRST NOTICE OF LOSS)	FNOL ID	First Notice of Event ID	VARCHAR	10	N/A	Yes	No	PK	Unique	FNOL-9999
	Loss Type ID	Loss Type ID	VARCHAR	10	N/A	Yes	No	FK	Must exist in Loss_Type table	LOSSID-9999
	Lost Cause	Cause of Loss	VARCHAR	150	Text	Yes	No	-	Text only	Self Damage
	Severity	Level of damage	VARCHAR	150	Text	Yes	No	-	Text only	Medium
	State Code	Code of the state	VARCHAR	5	Text	Yes	No	FK	Alphabetic characters only	AB
LOSS TYPE	Loss Type ID	Loss Type ID	VARCHAR	10	N/A	Yes	No	PK	Must exist in Loss_Type table	LOSSID-9999
	Loss Type Name	Type of damage	VARCHAR	150	Text	Yes	No	-	Text only	Vehicle damage
	Loss Category	Cause of Loss	VARCHAR	150	Text	Yes	No	-	Text only	Hit Deer
FINANCIAL	Financial ID	Financial Record Identifier	INT	10	N/A	Yes	No	PK	Unique	1001
	Payment Type	Type of Payment	VARCHAR	50	Text	Yes	No	-	Enum: [Cash, Card, Transfer]	Cash
	Payment Amount	Amount Paid	DECIMAL	10,2	N/A	Yes	No	-	Positive amount	1000.00
	Payment Status	Payment Update	BOOLEAN	N/A	N/A	Yes	No	-	Paid/Unpaid	Paid
	Deductible Applied	Deductible Applied Indicator	BOOLEAN	N/A	N/A	Yes	No	-	True/False	TRUE
POLICY	Policy ID	Policy Identifier	VARCHAR	10	N/A	Yes	No	PK	Unique	POLID-9999
	Policy Number	Policy Number	VARCHAR	20	N/A	Yes	No	-	Unique	POL-XYZ5H6
	Active	Policy is active or inactive	BOOLEAN	N/A	N/A	Yes	No	-	Active/Inactive	Active
	Holder Name	Policy Holder's Name	VARCHAR	100	Text	Yes	No	-	Text only	Sowmya Achar
	Policy Coverage ID	Coverage ID	VARCHAR	10	N/A	Yes	No	FK	Must exist in Policy_Coverage table	PCovID-9999
	Covered People ID	Unique identifier for person covered	VARCHAR	10	N/A	Yes	No	FK	Must be unique	CovID-9999
	Vehicle ID	Unique identifier for vehicle	VARCHAR	20	N/A	Yes	No	FK	Must be unique	AZ-345-PQR
POLICY COVERAGE	Policy Coverage ID	Coverage ID	VARCHAR	10	N/A	Yes	No	PK	Unique	PCovID-9999
	Coverage Name	Coverage Person Name	VARCHAR	20	Text	Yes	No	-	Text only	Vishnu Achar
	Covered Amount	Amount Covered	DECIMAL	10,2	N/A	Yes	No	-	Positive amount	10000
	Deductible	Remaining amount	DECIMAL	10,2	N/A	Yes	No	-	Positive amount	2000
VEHICLE DETAILS	Vehicle ID	Unique identifier for vehicle	VARCHAR	20	N/A	Yes	No	PK	Must be unique	AZ-345-PQR
	Vehicle Name	Name of the vehicle	VARCHAR	50	N/A	Yes	No	-	Alphanumeric	Toyota Camry
	Model	Model name of vehicle	VARCHAR	30	N/A	Yes	No	-	Alphanumeric	Camry
	Year Manufactured	Year manufactured	INT	4	YYYY	Yes	No	-	Year between 1900-Current Year	2020
	Color	Color of vehicle	VARCHAR	20	N/A	No	Yes	-	Alphanumeric	Blue
	Covered People ID	Unique identifier for person covered	VARCHAR	10	N/A	Yes	No	PK	Must be unique	CovID-9999

Entity Generation and Data Entry:

Overview:

The Claim Management System data model has been designed with the following tables, related through the relationships, namely: Claim, Policy, PolicyCoverage, CoveredPeople, VehicleDetails, History, Employer, Exposure, FNOL, Financial, and LossType. While implementing such tables, foreign key constraints should be defined to ensure that the data integrity is maintained and wrong data cannot be inserted in the tables. A foreign key value in the child table must match a primary key value of the associated parent table. This ensures consistency and reliability of information in the system. Resultantly, postings on the tables are done in an organized manner such that errors are avoided and successful execution of the SQL queries are allowed.

To create a database:

```
CREATE DATABASE IF NOT EXISTS CLAIM_MANAGEMENT_SYSTEM;
```

1) Entity Generation and Data Entry for Table Claim:

Query:

```
CREATE TABLE Claim
(
        claim_Number VARCHAR(30),
   policy_id VARCHAR(30),
   history_id VARCHAR(30),
   exposure_id VARCHAR(30),
   FNOL_ID VARCHAR(30),
   claimant_Fname VARCHAR(255),
   claimant_Lname VARCHAR(255),
   claim_State VARCHAR(10),
PRIMARY KEY(claim_Number),
FOREIGN KEY(policy_id) REFERENCES Policy(policy_id) ON DELETE CASCADE,
FOREIGN KEY(history_id) REFERENCES History(history_id) ON DELETE CASCADE,
FOREIGN KEY(exposure_id) REFERENCES Exposure(exposure_id) ON DELETE CASCADE,
FOREIGN KEY(FNOL_ID) REFERENCES FNOL(FNOL_ID) ON DELETE CASCADE
);
```

Explanation:

The Claim table will store all the information about the insurance claim and provides the required association among a policy, exposure, history, and First Notice of Loss record. This table picks up all the basic information of the claim, like the claim number, which is the unique identifier of the claim, the first and last name of the claimant, and the current status, whether the case is open or closed. The Claim table follows a very organized structure, which can enforce tracking every aspect of a claim by ensuring data integrity and hassle-free access through foreign keys referencing other tables that will be used for the comprehensive claim processing.

Command Used to Load the data from csv file to Claim table

```
LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/ Claim.csv'
INTO TABLE Claim
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

Data Ingestion:

The claim table data will be loaded from the CSV file using the LOAD DATA INFILE command, which introduces speed and accuracy in claiming data.

Screenshot:

```
172 •   select * from claim;
173
174
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

claim_Number	policy_id	history_id	exposure_id	FNOL_ID	claimant_Fname	claimant_Lname	claim_State
CLAIM-1235	POLID-1254	212236	EXPID-5678	FNOL-5678	Chris	Brown	INACTIVE
CLAIM-1236	POLID-1239	235181	EXPID-9012	FNOL-9012	Casey	Moore	ACTIVE
CLAIM-1237	POLID-1256	245826	EXPID-3456	FNOL-3456	Jane	Brown	ACTIVE
CLAIM-1238	POLID-1246	234387	EXPID-7890	FNOL-7890	Morgan	Jones	ACTIVE
CLAIM-1239	POLID-1257	229059	EXPID-7891	FNOL-7891	Jane	Williams	ACTIVE
CLAIM-1240	POLID-1250	197701	EXPID-7892	FNOL-7892	Casey	Miller	ACTIVE
CLAIM-1241	POLID-1255	198059	EXPID-7893	FNOL-7893	Pat	Moore	ACTIVE
CLAIM-1242	POLID-1256	222372	EXPID-7894	FNOL-7894	Jamie	Williams	ACTIVE
CLAIM-1243	POLID-1257	249630	EXPID-7895	FNOL-7895	Chris	Moore	ACTIVE
CLAIM-1244	POLID-1258	215194	EXPID-7896	FNOL-7896	Jordan	Brown	ACTIVE
CLAIM-1245	POLID-1234	229327	EXPID-7897	FNOL-7897	Chris	Johnson	ACTIVE
CLAIM-1246	POLID-1254	231966	EXPID-7898	FNOL-7898	Taylor	Smith	ACTIVE
CLAIM-1247	POLID-1239	204302	EXPID-7899	FNOL-7899	Casey	Jones	ACTIVE
CLAIM-1248	POLID-1256	208188	EXPID-7900	FNOL-7900	John	Davis	INACTIVE
CLAIM-1249	POLID-1246	214876	EXPID-7901	FNOL-7901	Chris	Miller	INACTIVE
CLAIM-1250	POLID-1257	223278	EXPID-7902	FNOL-7902	Jamie	Williams	INACTIVE
CLAIM-1251	POLID-1250	234254	EXPID-7903	FNOL-7903	Jamie	Wilson	INACTIVE
CLAIM-1252	POLID-1255	197788	EXPID-7904	FNOL-7904	Casey	Williams	ACTIVE
CLAIM-1253	POLID-1256	243725	EXPID-7905	FNOL-7905	Jordan	Williams	ACTIVE
CLAIM-1254	POLID-1257	239769	EXPID-7906	FNOL-7906	Morgan	Taylor	ACTIVE
CLAIM-1255	POLID-1258	209286	EXPID-7907	FNOL-7907	Pat	Taylor	ACTIVE
CLAIM-1256	POLID-1234	241246	EXPID-7908	FNOL-7908	John	Taylor	INACTIVE
CLAIM-1257	POLID-1254	228328	EXPID-7909	FNOL-7909	Jordan	Smith	INACTIVE

claim 20

2) Entity Generation and Data Entry for Table Policy:

Query:

```
CREATE TABLE Policy
(
    policy_id VARCHAR(30),
   policyNumber VARCHAR(30),
   active varchar(10),
   policyHolderName VARCHAR(255),
   policyCoverage_id VARCHAR(30),
   coveredPeople_id VARCHAR(30),
   vehicle_id VARCHAR(30),
PRIMARY KEY(policy_id),
FOREIGN KEY(policyCoverage_id) REFERENCES policyCoverage(policyCoverage_id) ON DELETE CASCADE,
FOREIGN KEY(coveredPeople_id) REFERENCES coveredPeople(coveredPeople_id) ON DELETE CASCADE,
FOREIGN KEY(vehicle_id) REFERENCES vehicleDetails(vehicle_id) ON DELETE CASCADE
);
```

Explanation:

The Policy table is an inclusive table that comprises all data regarding insurance policies, having a unique policy ID, policy number, policyholder's name, and the status of the policy. It connects further, through foreign key relations, to other related entities like policy coverage, covered individuals, and vehicles covered under that policy.

The policyholder can strongly represent his or her insured property and persons by maintaining these relationships enabled through the Policy table. This will help in accurate identification of eligibility and verification of coverage of events of claims and thus is indispensable in determining the range of protection granted under each policy.

Command Used to Load the data from csv file to Policy table

```
LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/ Policy.csv'
INTO TABLE Policy
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

Data Ingestion:

The CSV file will import all the details about the policy directly into the database via a bulk data entry LOAD DATA INFILE command for the Policy table without manual errors.

Screenshot:

```
71
72 •   select * from policy;
```

esult Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

policy_id	policyNumber	active	policyHolderName	policyCoverage_id	coveredPeople_id	vehicle_id
POLID-1234	POL-FPN5AH	1	John Taylor	PCovID-7890	CovID-7908	CA-456-DEF
POLID-1234	POL-8MR0VI	1	Chris Johnson	PCovID-7891	CovID-7897	NJ-234-DEF
POLID-1239	POL-S2OFH3	1	Casey Jones	PCovID-5678	CovID-7899	MA-890-XYZ
POLID-1239	POL-HZYYEK	1	Casey Smith	PCovID-7891	CovID-7910	NY-101-XYZ
POLID-1239	POL-WNPCUO	1	Casey Moore	PCovID-9012	CovID-9012	FL-789-DEF
POLID-1246	POL-TLAWBF	0	Chris Miller	PCovID-3456	CovID-7901	CO-456-ABC
POLID-1246	POL-IZKXTE	1	Morgan Jones	PCovID-7890	CovID-7890	TX-234-XYZ
POLID-1250	POL-VS62ZA	0	Casey Miller	PCovID-1234	CovID-7892	OH-890-ABC
POLID-1250	POL-PNPUVY	1	Jamie Wilson	PCovID-7891	CovID-7903	MN-101-DEF
POLID-1254	POL-81KL5M	1	Taylor Smith	PCovID-1234	CovID-7898	NC-567-ABC
POLID-1254	POL-QJ2HLW	0	Chris Brown	PCovID-5678	CovID-5678	CA-456-XYZ
POLID-1254	POL-QHCVX4	1	Jordan Smith	PCovID-7891	CovID-7909	FL-789-ABC
POLID-1255	POL-U554HD	1	Casey Williams	PCovID-1234	CovID-7904	MD-234-ABC
POLID-1255	POL-RQENSX	1	Pat Moore	PCovID-5678	CovID-7893	MI-123-XYZ
POLID-1256	POL-FT4Q0I	1	Jane Brown	PCovID-3456	CovID-3456	NY-101-ABC
POLID-1256	POL-P888T8	0	Jordan Williams	PCovID-5678	CovID-7905	MO-567-XYZ
POLID-1256	POL-130HAB	1	John Davis	PCovID-7891	CovID-7900	WA-123-DEF
POLID-1256	POL-K1BVTV	0	Jamie Johnson	PCovID-9012	CovID-7894	IL-456-DEF
POLID-1257	POL-3XDFVL	1	Chris Moore	PCovID-3456	CovID-7895	GA-789-ABC
POLID-1257	POL-3TL6ZB	1	Jamie Johnson	PCovID-7890	CovID-7902	TN-789-XYZ
POLID-1257	POL-A26IWL	0	Jane Williams	PCovID-7891	CovID-7891	PA-567-DEF
POLID-1257	POL-ZSCCED	0	Morgan Taylor	PCovID-9012	CovID-7906	IN-890-ABC
POLID-1258	POL-OGBQKH	0	Pat Taylor	PCovID-3456	CovID-7907	AZ-123-XYZ

licy 21 ×

3) Entity Generation and Data Entry for Table PolicyCoverage:

Query:

```
CREATE TABLE PolicyCoverage
(
        policyCoverage_id VARCHAR(30),
   coverageName VARCHAR(30),
   coveredAmount FLOAT NOT NULL,
   deductible FLOAT,
PRIMARY KEY(policyCoverage_id)
);
```

Explanation:

PolicyCoverage table focuses on the detailed coverage entailed in an insurance policy. It captures a unique identifier for each category of coverage, the naming of coverage type-say, collision or liability-the monetary amount covered under the policy, and any applicable deductibles. This table will make sure the monetary aspect of the policy is well recorded for clarity on what is included in the cover and allows the claim assessment to be precisely made.

It plays a critical role in the claims process in that it sets the financial limit of policy, payment.

Command Used to Load the data from csv file to PolicyCoverage table

```
LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/
PolicyCoverage.csv'

INTO TABLE PolicyCoverage

FIELDS TERMINATED BY ','

ENCLOSED BY '"'

LINES TERMINATED BY '\n'

IGNORE 1 ROWS;
```

Data Ingestion:

This command uses the LOAD DATA INFILE command, which imports the information of coverages for the PolicyCoverage table efficiently via a CSV file.

Screenshot:

```
172 •   select * from policycoverage;
```

Result Grid | Filter Rows: | Edit: | Export/I

policyCoverage_id	coverageName	coveredAmount	deductible
PCovID-1234	Liability	28641.69	1920.14
PCovID-3456	Uninsured Motorist	29495.41	125.55
PCovID-5678	Collision	84243.92	720.88
PCovID-7890	Personal Injury Protection	55324.8	1693.53
PCovID-7891	Third party Liability	41615.24	583.02
PCovID-9012	Comprehensive	17770.7	1886.38

4) Entity Generation and Data Entry for Table CoveredPeople:

Query:

```
CREATE TABLE CoveredPeople
(
        coveredPeople_id VARCHAR(30),
   firstName VARCHAR(255),
   lastName VARCHAR(255),
   relation VARCHAR(255),
PRIMARY KEY(coveredPeople_id)
);
```

Explanation:

The CoveredPeople table contains information about the people covered under a specific policy, first and last name, and their relation to the policyholder, like spouse or dependent. Each record has a unique key that identifies the covered peoples, 'Covered People ID', which links to the 'Policy' table. This helps the insurance company pinpoint exactly the people that have been covered under one policy and manage the claims related to these people.

Command Used to Load the data from csv file to CoveredPeople table

```
LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/
CoveredPeople.csv'
INTO TABLE CoveredPeople
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

Data Ingestion:

Importing data into the CoveredPeople table is achieved using a CSV file through the LOAD DATA INFILE command, which allows the structured inputting of people's data.

Screenshot:

```
172 •   select * from coveredpeople;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

coveredPeople_id	firstName	lastName	relation
CovID-1234	Chris	Davis	Parent
CovID-3456	Jane	Brown	Parent
CovID-5678	Chris	Jones	Parent
CovID-7890	John	Wilson	Child
CovID-7891	Morgan	Moore	Self
CovID-7892	Jordan	Williams	Child
CovID-7893	Jamie	Moore	Child
CovID-7894	Morgan	Moore	Spouse
CovID-7895	Pat	Smith	Parent
CovID-7896	Taylor	Wilson	Child
CovID-7897	Pat	Wilson	Parent
CovID-7898	Chris	Brown	Self
CovID-7899	Jamie	Jones	Parent
CovID-7900	Jordan	Miller	Self
CovID-7901	Jamie	Miller	Parent
CovID-7902	Chris	Johnson	Child
CovID-7903	Jordan	Davis	Self
CovID-7904	Chris	Davis	Self
CovID-7905	Jamie	Brown	Parent
CovID-7906	Jordan	Wilson	Spouse
CovID-7907	Chris	Brown	Spouse
CovID-7908	Taylor	Jones	Self
CovID-7909	Casey	Smith	Child

coveredpeople23 ×

5) Entity Generation and Data Entry for Table VehicleDetails:

Query:

```
CREATE TABLE VehicleDetails
(
        vehicle_id VARCHAR(30),
    vehicleName VARCHAR(30),
    model VARCHAR(20),
    yearManufactured DATE,
    color VARCHAR(20),
```

```
PRIMARY KEY(vehicle_id)
);
```

Explanation:

The VehicleDetails table summarizes the information on the vehicle covered under a particular policy. It links up the vehicle name, model, and year of production with its color through the vehicle identification number. It is essential for any claim about a policy that covers automotive, as it allows for proper identification and valuation of the vehicle.

Command Used to Load the data from csv file to VehicleDetails table

```
LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/
VehicleDetails.csv'
INTO TABLE VehicleDetails
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

Data Ingestion:

The data for vehicles are populated to the VehicleDetails table using the import of comma-separated values via the LOAD DATA INFILE command.

Screenshot:

```
172 •   select * from vehicledetails;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

vehicle_id	vehicleName	model	yearManufactured	color
AZ-123-ABC	Toyota Corolla	Coupe	2022-01-01	Black
AZ-123-XYZ	Tesla Model 3	SUV	1994-01-01	Blue
CA-456-DEF	Honda Civic	Coupe	2006-01-01	Grey
CA-456-XYZ	Tesla Model 3	Truck	2005-01-01	Blue
CO-456-ABC	Ford Focus	SUV	2018-01-01	White
FL-789-ABC	Toyota Corolla	SUV	2010-01-01	Silver
FL-789-DEF	Ford Focus	Coupe	2010-01-01	Grey
GA-789-ABC	Chevrolet Malibu	SUV	2022-01-01	Black
IL-456-DEF	Ford Focus	Coupe	1995-01-01	Black
IN-890-ABC	Ford Focus	SUV	2006-01-01	Silver
MA-890-XYZ	Toyota Corolla	Sedan	2020-01-01	Blue
MD-234-ABC	Chevrolet Malibu	SUV	1998-01-01	Silver
MI-123-XYZ	Ford Focus	Sedan	2022-01-01	Red
MN-101-DEF	Honda Civic	Truck	1993-01-01	Red
MO-567-XYZ	Honda Civic	Sedan	2012-01-01	Blue
NC-567-ABC	Honda Civic	Sedan	2023-01-01	White
NJ-234-DEF	Tesla Model 3	Truck	2020-01-01	Grey
NY-101-ABC	Toyota Corolla	SUV	2014-01-01	Red
NY-101-XYZ	Chevrolet Malibu	Truck	2000-01-01	Red
OH-890-ABC	Chevrolet Malibu	SUV	2007-01-01	Red
PA-567-DEF	Ford Focus	Truck	2010-01-01	Grey
TN-789-XYZ	Chevrolet Malibu	Coupe	2022-01-01	Red
TX-234-XYZ	Honda Civic	Truck	1991-01-01	Grey

vehicledetails 24

6) Entity Generation and Data Entry for Table History:

Query:

```
CREATE TABLE History
(
    history_id VARCHAR(30),
   employee_id VARCHAR(30),
   created DATE,
   edited DATE,
   deleted DATE,
PRIMARY KEY(history_id),
FOREIGN KEY(employee_id) REFERENCES employer(employee_id) ON DELETE CASCADE
);
```

Explanation:

The History table will track all significant changes made to either claims or policies, including creating, editing, and delete date/time stamps. It will also identify what employee has made a certain change by referring back to the Employer table. This provides full audit trail capabilities that can be very useful when managing accountability regarding policies and claims.

Command Used to Load the data from csv file to History table

```
LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/ History.csv'
INTO TABLE History
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

Data Ingestion:

Records in the History table are populated from a CSV file using the LOAD DATA INFILE command.

Screenshot:

```
172 •   select * from History;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Cont

history_id	employee_id	created	edited	deleted
194115	18ICP104	2024-05-31 18:05:00	2023-05-25 23:56:00	2023-09-15 14:29:00
197701	17ICP215	2024-01-04 21:40:00	2024-10-09 20:39:00	2024-09-03 22:49:00
197788	22ICP206	2024-01-09 22:45:00	2024-02-08 15:56:00	2024-05-22 11:25:00
198059	18ICP218	2023-11-09 12:20:00	2023-03-13 02:13:00	2023-09-08 13:06:00
204302	19ICP098	2024-02-17 09:26:00	2024-06-05 20:32:00	2023-05-19 09:49:00
208188	20ICP024	2024-08-06 02:37:00	2023-07-22 06:46:00	2024-09-24 04:58:00
209286	16ICP098	2023-05-25 13:06:00	2023-11-28 18:33:00	2023-12-01 11:48:00
211342	24ICP104	2024-09-08 00:00:00	2023-05-03 10:29:00	2023-03-02 04:32:00
212236	21ICP113	2024-02-20 01:02:00	2024-10-01 23:28:00	2024-06-20 12:27:00
214876	16ICP060	2024-06-05 20:44:00	2023-02-14 05:21:00	2023-02-11 09:51:00
215194	24ICP111	2023-08-08 02:30:00	2023-09-07 18:01:00	2023-11-18 06:43:00
222372	22ICP329	2024-06-29 21:26:00	2024-08-16 01:44:00	2023-08-25 05:29:00
223278	15ICP222	2023-06-17 23:38:00	2023-12-19 07:41:00	2023-11-23 17:11:00
228328	19ICP006	2023-06-21 14:46:00	2023-11-30 21:29:00	2024-04-14 16:51:00
229059	15ICP003	2024-07-26 16:48:00	2024-01-06 10:56:00	2023-06-02 14:37:00
229327	21ICP006	2024-01-13 12:57:00	2023-03-24 06:31:00	2023-08-09 23:26:00
231966	17ICP234	2024-10-07 00:03:00	2024-08-26 00:23:00	2023-10-26 15:27:00
234254	15ICP262	2024-08-17 00:25:00	2023-05-15 21:31:00	2024-06-13 10:56:00
234387	18ICP216	2023-08-16 00:53:00	2024-04-17 21:13:00	2023-12-11 01:41:00
235181	20ICP002	2023-01-03 04:52:00	2024-02-27 18:57:00	2024-02-10 12:03:00
239769	22ICP024	2023-07-02 15:55:00	2023-08-26 04:27:00	2023-05-30 00:34:00
241246	20ICP234	2023-10-16 05:31:00	2023-12-18 18:26:00	2023-05-23 19:33:00
243725	23ICP060	2024-03-16 00:19:00	2024-09-28 09:35:00	2023-11-22 13:26:00

History 25 ×

7) Entity Generation and Data Entry for Table Employer:

Query:

```
CREATE TABLE Employer
(
        employee_id VARCHAR(30),
   firstName VARCHAR(255),
   lastName VARCHAR(255),
   email VARCHAR(255),
   phoneNumber INT,
 PRIMARY KEY(employee_id),
 UNIQUE KEY(email),
 UNIQUE KEY(phoneNumber)
);
```

Explanation:

The Employer table stores much information about the employees who contribute to the claim management process. It contains Employee ID, Firstname, Lastname, Email address, and Telephone number. Unique constraints for both email and phone numbers prevent duplication entries. This table allows this application to implement the required role-based access and identify which employee is responsible for various functions throughout this application.

Command Used to Load the data from csv file to Employer table

```
LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/ Employer.csv'
INTO TABLE Employer
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

Data Ingestion:

The LOAD DATA INFILE command will import the data about employees in the Employer table from a CSV.

Screenshot:

```
172 •   select * from employer;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

employee_id	firstName	lastName	email	phoneNumber
14ICP114	Casey	Wilson	casey.wilson@company.com	9689209693
15ICP003	Taylor	Williams	taylor.williams@outlook.com	9177369447
15ICP222	Alex	Miller	alex.miller@gmail.com	3642918421
15ICP262	Morgan	Johnson	morgan.johnson@outlook.com	9268035260
16ICP060	John	Smith	john.smith@yahoo.com	2313572190
16ICP098	Jane	Davis	jane.davis@gmail.com	2254198924
17ICP215	Jamie	Taylor	jamie.taylor@gmail.com	1382694599
17ICP234	Chris	Smith	chris.smith@company.com	9125745949
18ICP104	John	Miller	john.miller@outlook.com	3502572285
18ICP216	Jamie	Davis	jamie.davis@yahoo.com	1823177251
18ICP218	Pat	Williams	pat.williams@yahoo.com	2117472698
19ICP006	Jamie	Smith	jamie.smith@company.com	2019644141
19ICP098	Pat	Miller	pat.miller@company.com	6423143597
20ICP002	Morgan	Davis	morgan.davis@gmail.com	6386775582
20ICP024	Jordan	Moore	jordan.moore@gmail.com	4125227916
20ICP234	Jane	Johnson	jane.johnson@outlook.com	3772339480
21ICP006	Pat	Davis	pat.davis@gmail.com	5892482814
21ICP009	Pat	Miller	pat.miller@yahoo.com	6449204532
21ICP113	Casey	Miller	casey.miller@outlook.com	2888585032
22ICP024	Jordan	Wilson	jordan.wilson@gmail.com	2527468070
22ICP206	Jane	Jones	jane.jones@outlook.com	1515494068
22ICP329	Morgan	Jones	morgan.jones@company.com	3233808350
23ICP060	Alex	Moore	alex.moore@outlook.com	1781184028

8) Entity Generation and Data Entry for Table Exposure:

Query:

```
CREATE TABLE Exposure
(
   exposure_id VARCHAR(30),
   exposureName VARCHAR(255),
   policyCoverage_id VARCHAR(30),
   FNOL_id VARCHAR(30),
   financials_id VARCHAR(30),
   exposureState VARCHAR(10),
PRIMARY KEY(exposure_id),
FOREIGN KEY(policyCoverage_id) REFERENCES PolicyCoverage(policyCoverage_id) ON DELETE CASCADE,
FOREIGN KEY(FNOL_id) REFERENCES FNOL(FNOL_id) ON DELETE CASCADE,
FOREIGN KEY(financials_id) REFERENCES Financial(financial_id) ON DELETE CASCADE
);
```

Explanation:

The Exposure table holds particular risk or losses to be insured under a policy; examples include the name of the exposure, current state of exposure, and links to financials, FNOL, and policy coverage. It helps in monitoring each exposure involved in claims to make sure all possible risks are mitigated and assessed properly during the course of a claim.

Command Used to Load the data from csv file to Exposure table

```
LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/ Exposure.csv'
INTO TABLE Exposure
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

Data Ingestion:

The information in the Exposure table will be added by importing the data from a CSV file using the LOAD DATA INFILE command.

Screenshot:

```
172 •   select * from exposure;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

exposure_id	exposureName	policyCoverage_id	FNOL_id	financials_id	exposureState
EXPID-1234	Collision	PCovID-1234	FNOL-1234	FinID-1234	Open
EXPID-3456	Comprehensive	PCovID-3456	FNOL-3456	FinID-3456	Open
EXPID-5678	Comprehensive	PCovID-5678	FNOL-5678	FinID-5678	Close
EXPID-7890	Collision	PCovID-7890	FNOL-7890	FinID-7890	Close
EXPID-7891	Collision	PCovID-1234	FNOL-1234	FinID-7891	Open
EXPID-7892	Comprehensive	PCovID-5678	FNOL-5678	FinID-7892	Open
EXPID-7893	Collision	PCovID-9012	FNOL-9012	FinID-7893	Open
EXPID-7894	Person Liability	PCovID-3456	FNOL-3456	FinID-7894	Close
EXPID-7895	Person Liability	PCovID-7890	FNOL-7890	FinID-7895	Open
EXPID-7896	Comprehensive	PCovID-1234	FNOL-1234	FinID-7896	Open
EXPID-7897	Collision	PCovID-5678	FNOL-5678	FinID-7897	Close
EXPID-7898	Comprehensive	PCovID-9012	FNOL-9012	FinID-7898	Open
EXPID-7899	Person Liability	PCovID-3456	FNOL-3456	FinID-7899	Close
EXPID-7900	Collision	PCovID-7890	FNOL-7890	FinID-7900	Open
EXPID-7901	Comprehensive	PCovID-1234	FNOL-1234	FinID-7901	Open
EXPID-7902	Person Liability	PCovID-5678	FNOL-5678	FinID-7902	Close
EXPID-7903	Comprehensive	PCovID-9012	FNOL-9012	FinID-7903	Open
EXPID-7904	Person Liability	PCovID-3456	FNOL-3456	FinID-7904	Close
EXPID-7905	Collision	PCovID-7890	FNOL-7890	FinID-7905	Open
EXPID-7906	Comprehensive	PCovID-1234	FNOL-1234	FinID-7906	Open
EXPID-7907	Person Liability	PCovID-5678	FNOL-5678	FinID-7907	Close
EXPID-7908	Comprehensive	PCovID-9012	FNOL-9012	FinID-7908	Close
EXPID-7909	Person Liability	PCovID-3456	FNOL-3456	FinID-7909	Open

exposure 27 ×

9) Entity Generation and Data Entry for Table FNOL:

Query:

```
CREATE TABLE FNOL
(
        FNOL_id VARCHAR(30),
   lossType_id VARCHAR(30),
   lossCause VARCHAR(255),
   severity VARCHAR(10),
PRIMARY KEY(FNOL_id),
FOREIGN KEY(lossType_id) REFERENCES LossType(lossType_id) ON DELETE CASCADE
);
```

Explanation:

First Notice of Loss Table First reporting of a loss or incident that may involve a claim includes type of loss, cause of loss, and the degree of severity. The table would relate back to the LossType table as it would categorize incidents so that claims could begin processing as quickly as possible.

Command Used to Load the data from csv file to FNOL table

```
LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/ FNOL.csv'
INTO TABLE FNOL
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

Data Ingestion:

The information for the FNOL table is fed to the database through the LOAD DATA INFILE command via the CSV file.

Screenshot:

```
select * from FNOL;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

FNOL_id	lossType_id	lossCause	severity	StateCode
FNOL-1234	LOSSID-1234	SELF DAMAGE	MEDIUM	CA
FNOL-3456	LOSSID-3456	SELF DAMAGE	LOW	FL
FNOL-5678	LOSSID-5678	SELF DAMAGE	LOW	NY
FNOL-7890	LOSSID-7890	THIRD PARTY	MEDIUM	PA
FNOL-7891	LOSSID-1234	SELF DAMAGE	MEDIUM	IL
FNOL-7892	LOSSID-5678	THIRD PARTY	LOW	OH
FNOL-7893	LOSSID-9012	SELF DAMAGE	HIGH	GA
FNOL-7894	LOSSID-3456	THIRD PARTY	MEDIUM	MI
FNOL-7895	LOSSID-7890	THIRD PARTY	LOW	NJ
FNOL-7896	LOSSID-1234	SELF DAMAGE	MEDIUM	VA
FNOL-7897	LOSSID-5678	SELF DAMAGE	LOW	WA
FNOL-7898	LOSSID-9012	THIRD PARTY	MEDIUM	NC
FNOL-7899	LOSSID-3456	SELF DAMAGE	HIGH	MA
FNOL-7900	LOSSID-7890	THIRD PARTY	MEDIUM	CO
FNOL-7901	LOSSID-1234	THIRD PARTY	LOW	TN
FNOL-7902	LOSSID-5678	SELF DAMAGE	LOW	MN
FNOL-7903	LOSSID-9012	THIRD PARTY	MEDIUM	MD
FNOL-7904	LOSSID-3456	THIRD PARTY	LOW	MO
FNOL-7905	LOSSID-7890	SELF DAMAGE	HIGH	IN
FNOL-7906	LOSSID-1234	THIRD PARTY	LOW	AZ
FNOL-7907	LOSSID-5678	SELF DAMAGE	MEDIUM	CT
FNOL-7908	LOSSID-9012	THIRD PARTY	HIGH	KY
FNOL-7909	LOSSID-3456	SELF DAMAGE	HIGH	LA

FNOL 28

10) Entity Generation and Data Entry for Table Financial:

Query:

```
CREATE TABLE Financial
(
        financial_id VARCHAR(30),
   paymentType VARCHAR(255),
   paymentAmmount FLOAT NOT NULL,
   paymentStatus VARCHAR(10),
   deductibleApplied FLOAT,
PRIMARY KEY(financial_id)
);
```

Explanation:

The Financial table oversees all payment-related information linked to claims. It monitors the method of payment (for instance, cash or check), the amount disbursed, the status of the payment, and any deductibles that have been applied. This table guarantees clarity regarding the financial dimensions of claims processing and offers a comprehensive analysis of monetary transactions.

Command Used to Load the data from csv file to Financial table

```
LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/ Financial.csv'

INTO TABLE Financial

FIELDS TERMINATED BY ','

ENCLOSED BY '"'

LINES TERMINATED BY '\n'

IGNORE 1 ROWS;
```

Data Ingestion:

Loading Data into the financial table is populated with financial information from the CSV file by using a command called LOAD DATA INFILE.

Screenshot:

```
172 •   select * from financial;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

financial_id	paymentType	paymentAmmount	paymentStatus	deductibleApplied
FinID-1234	Card	123.45	Paid	20
FinID-3456	Card	34.56	Paid	50
FinID-5678	Cash	56.78	Unpaid	30
FinID-7890	Cash	78.9	Unpaid	60
FinID-7891	Transfer	12.34	Paid	70
FinID-7892	Card	56.78	Unpaid	80
FinID-7893	Cash	90.12	Paid	90
FinID-7894	Transfer	34.56	Unpaid	20
FinID-7895	Card	78.9	Paid	30
FinID-7896	Cash	12.34	Unpaid	40
FinID-7897	Transfer	56.78	Paid	50
FinID-7898	Card	90.12	Unpaid	60
FinID-7899	Cash	34.56	Paid	70
FinID-7900	Transfer	78.9	Unpaid	80
FinID-7901	Card	12.34	Paid	90
FinID-7902	Cash	56.78	Unpaid	20
FinID-7903	Transfer	90.12	Paid	30
FinID-7904	Card	34.56	Unpaid	40
FinID-7905	Cash	78.9	Paid	50
FinID-7906	Transfer	12.34	Unpaid	60
FinID-7907	Card	56.78	Paid	70
FinID-7908	Cash	90.12	Unpaid	80
FinID-7909	Transfer	34.56	Paid	90

financial 29

11) Entity Generation and Data Entry for Table LossType:

Query:

```
CREATE TABLE LossType
(
        lossType_id VARCHAR(30),
   lossTypeName VARCHAR(255),
   lossCategory VARCHAR(255),
PRIMARY KEY(lossType_id)
);
```

Explanation:

The LossType table categorizes various types of losses that an insurance policy might cover. The table will hold fields for the loss type name and category, making the classification and analysis of claims more orderly. The table will provide uniformity in loss classification and facilitate detailed reporting and analytics.

Command Used to Load the data from csv file to LossType table

```
LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/ LossType.csv'
INTO TABLE LossType
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

Data Ingestion:

Importing data into a database from the CSV file will be accomplished through using the LOAD DATA INFILE command.

Screenshot:

```
172 •   select * from lossType;
```

Result Grid | Filter Rows: | Edit:

lossType_id	lossTypeName	lossCategory
LOSSID-1234	Collision Damage	Auto
LOSSID-3456	Flood Damage	Personal
LOSSID-5678	Theft	Personal
LOSSID-7890	Hail Damage	Personal
LOSSID-9012	Hit-and-Run Damage	Auto

12) Entity Generation and Data Entry for Table LossType:

Query:

CREATE TABLE LossState

(

StateCode VARCHAR(5),

StateName VARCHAR(40),

PRIMARY KEY(StateCode)

);

Explanation:

The loss state table contains the state code and their respective names, this will used as a foreign key in the FNOL table which helps the user to understand where the loss happened.

Command Used to Load the data from csv file to LossType table

LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/ LossState.csv'

INTO TABLE LossState

FIELDS TERMINATED BY ','

ENCLOSED BY '"'

LINES TERMINATED BY '\n'

IGNORE 1 ROWS;

```
183 •   select * from lossstate;
```

Result Grid | Filter Rows: | Edit:

StateCode	StateName
AL	Alabama
AZ	Arizona
[illegible]	California
CO	Collarodo
CT	Connecticut
FL	Florida
[illegible]	Georgia
IL	Illinois
IN	Indiana
KY	Kentucky
LA	Loss Angelas
MA	Massachus...
MD	Maryland
MI	Michigan

lossstate41 ×

Data Ingestion:

Importing data into a database from the CSV file will be accomplished through using the LOAD DATA INFILE command.

Screenshot:

Data Retrieval and Simple Reports:

Overview:

Information extraction from raw data into insightful information is an essential part of doing business today, as it enables the organization to use the extracted information toward making informed decisions. This project involves formulating queries with the intent of stakeholders deriving necessary insights to improve processes and optimize all operations. Results from the analysis will allow management to decrease operation costs, increase profitability, and ultimately drive high customer satisfaction. This will involve explaining what the purpose of every query is, breaking down each simple SQL query, and the complete SQL SELECT statement. Attached also are screenshots of the result set of the queries to show the findings and help interpret data.

Data Analysis:

The advanced analysis gives more information on your database by explaining the relation, trends, and irregularities within the tables. The type of policies that have the highest claims per occurrence, top employees who handle claims, and causes of loss by state. Loss costs are reviewed by type of loss and state, whereas claims are viewed for duplication, severity, and lack of related exposures or policy information. The analysis also identifies the relative unused resources including the orphan policies with zero corresponding claims or financial values. Incorporating vehicle and policyholder data, the queries give ideas of operation performance and recommendations for improvement.

Analysis -1 :

This analysis helps us to learn the total number of the claims done by the policy holders. Which in turn helps us to identify the pattern and reassess the policy premium.

Query:

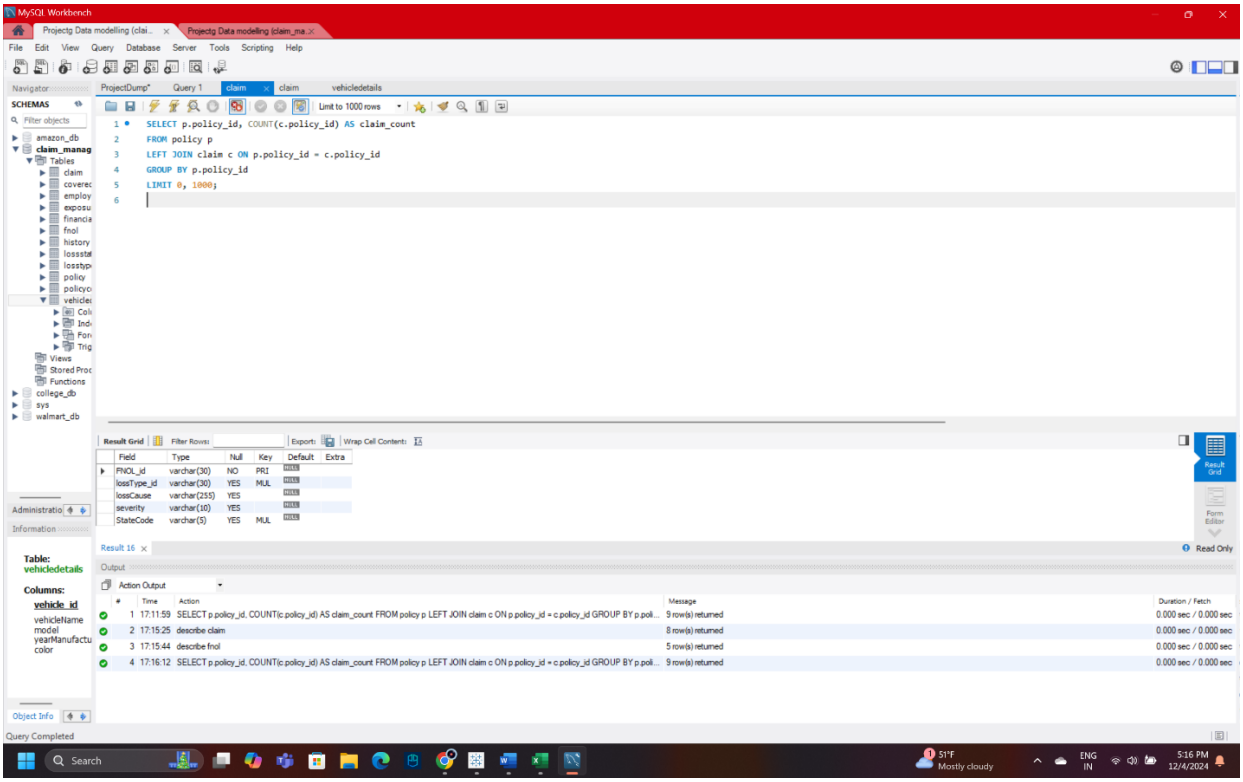
```
SELECT p.policy_id, COUNT(c.policy_id) AS claim_count

FROM policy p

LEFT JOIN claim c ON p.policy_id = c.policy_id

GROUP BY p.policy_id
```

LIMIT 0, 1000;

Explanation:

This query identifies the policies with more than one claim associated with them and displays them.

Result and screenshot:



Analysis – 2:

To find out the employees who processed most number of claims. This helps us to evaluate the employees and reward the best.

<u>Query:</u>

SELECT e.employee_id, e.firstName, e.lastName, COUNT(c.history_id) AS claims_processed

FROM employer e

JOIN history h ON e.employee_id = h.employee_id

JOIN claim c ON h.history_id = c.history_id

GROUP BY e.employee_id

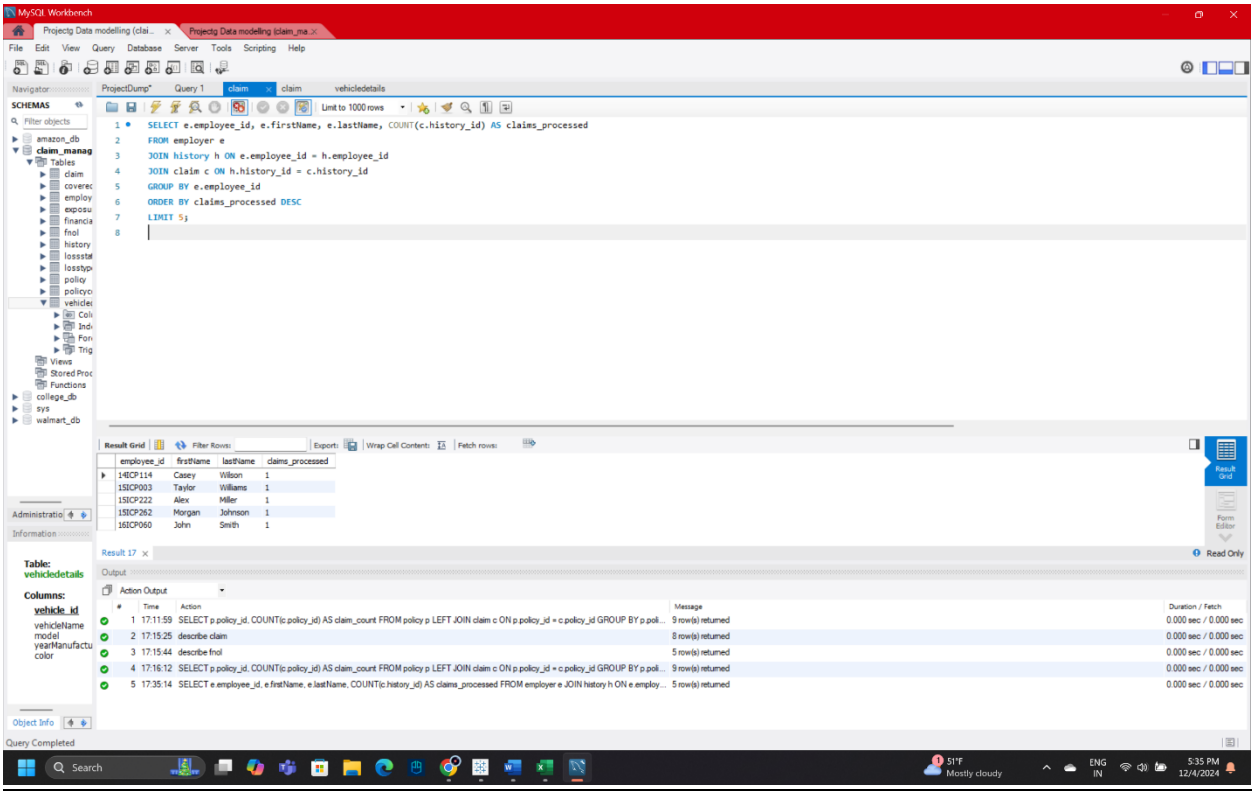
ORDER BY claims_processed DESC

LIMIT 5;

Explanation:

This query joins the tables of employers, claim and history. It identifies the employees while grouping them based on the employee.id. The order by helps us to determine the employee's in a descending order.

Result and Screenshot:



Analysis -3:

Understanding the claims which have missing exposure details are key to assess where the data is being entered with negligence. This helps us to resolve any problems that might arise in the future.

Query:

SELECT c.claim_Number, c.policy_id, c.exposure_id

FROM claim c

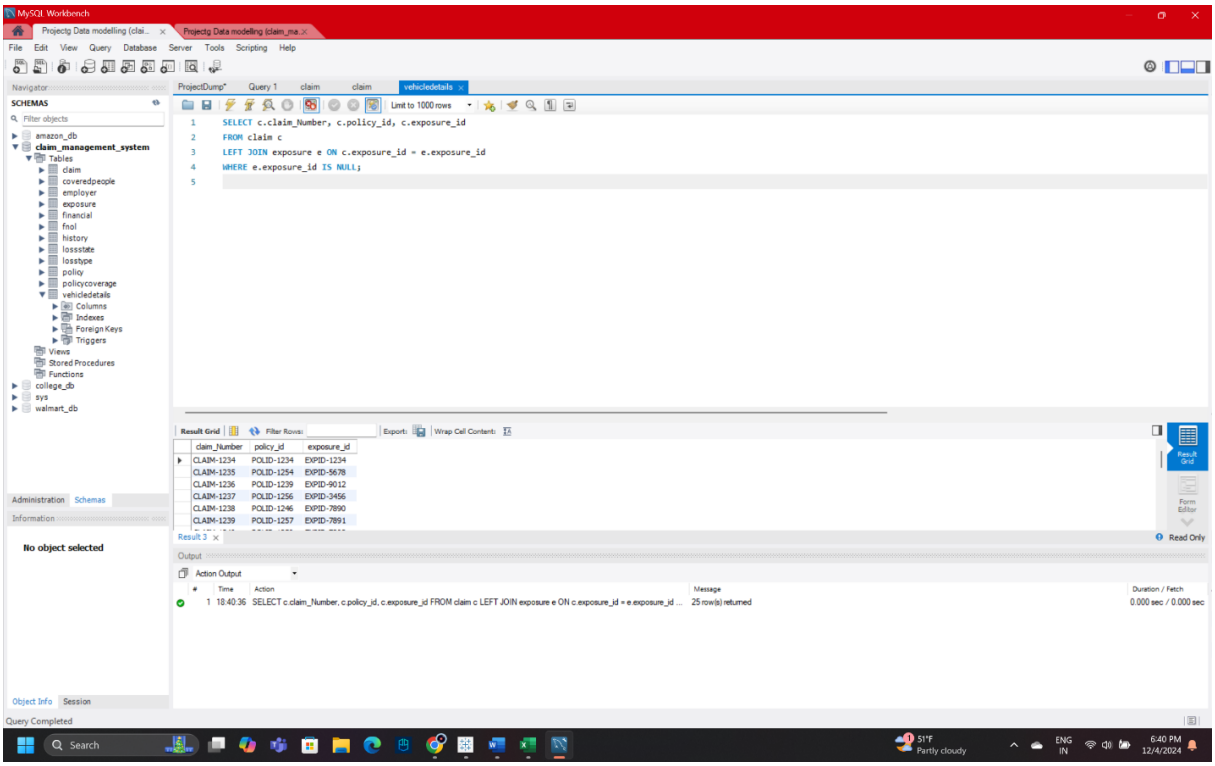
LEFT JOIN exposure e ON c.exposure_id = e.exposure_id

WHERE e.exposure_id IS NULL;

Explanation:

This query selects the claim number, policy ID, and associated exposure ID. This is possible by left joining the exposure onto the claims table. While also keeping a conditional statement WHERE. Which helps us to identify null IDs.

Result and Screenshot:



Analysis 4:

To analyze the distribution of claims based on the severity of the incident.

Query:

SELECT f.severity, COUNT(c.claim_Number) AS claim_count

FROM fnol f

JOIN claim c ON f.FNOL_id = c.FNOL_ID

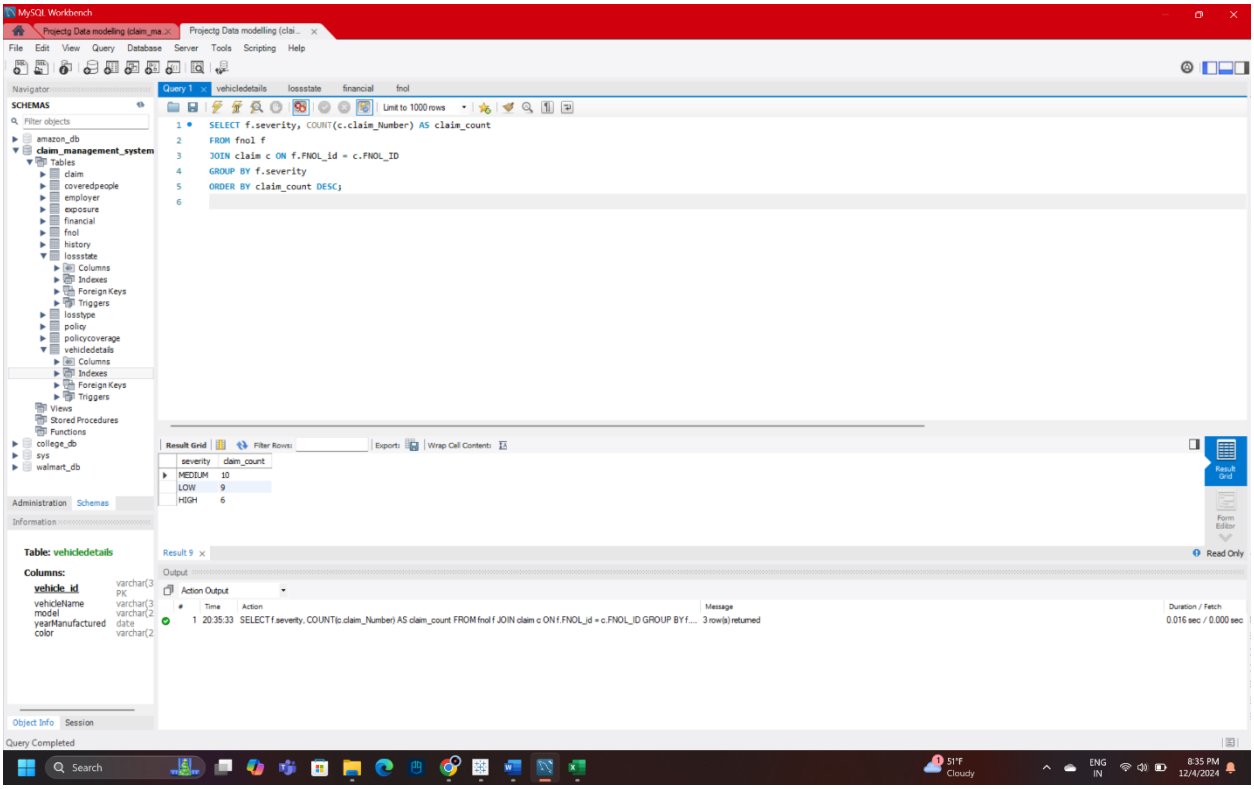
GROUP BY f.severity

ORDER BY claim_count DESC;

Explanation:

The query helps to determine the total count of claims in order to compare their distribution by the level of claimed severities. It initially, extracts severity levels from the 'fnol' table and tallys up related claims from the 'claim' table. It employs the `FNOL_id" to link the tables so that related records can be easily identified. After that, the results are partitioned with key `f.severity` to guarantee that each severity level will be aggregated with its claims. Lastly, it provides an interesting quantization as the severity level is arranged in descending order of the count of the claim. These identified criteria enable the prioritization of the claims to be addressed depending on their level of harm.

Result and Screenshot:



Analysis – 5:

To find active policies and associated claims for them.

Query:

```
SELECT p.policyNumber,
    p.policyHolderName,
    COUNT(c.claim_Number) AS claim_count
FROM policy p
```

LEFT JOIN claim c ON p.policy_id = c.policy_id

WHERE p.active = 1

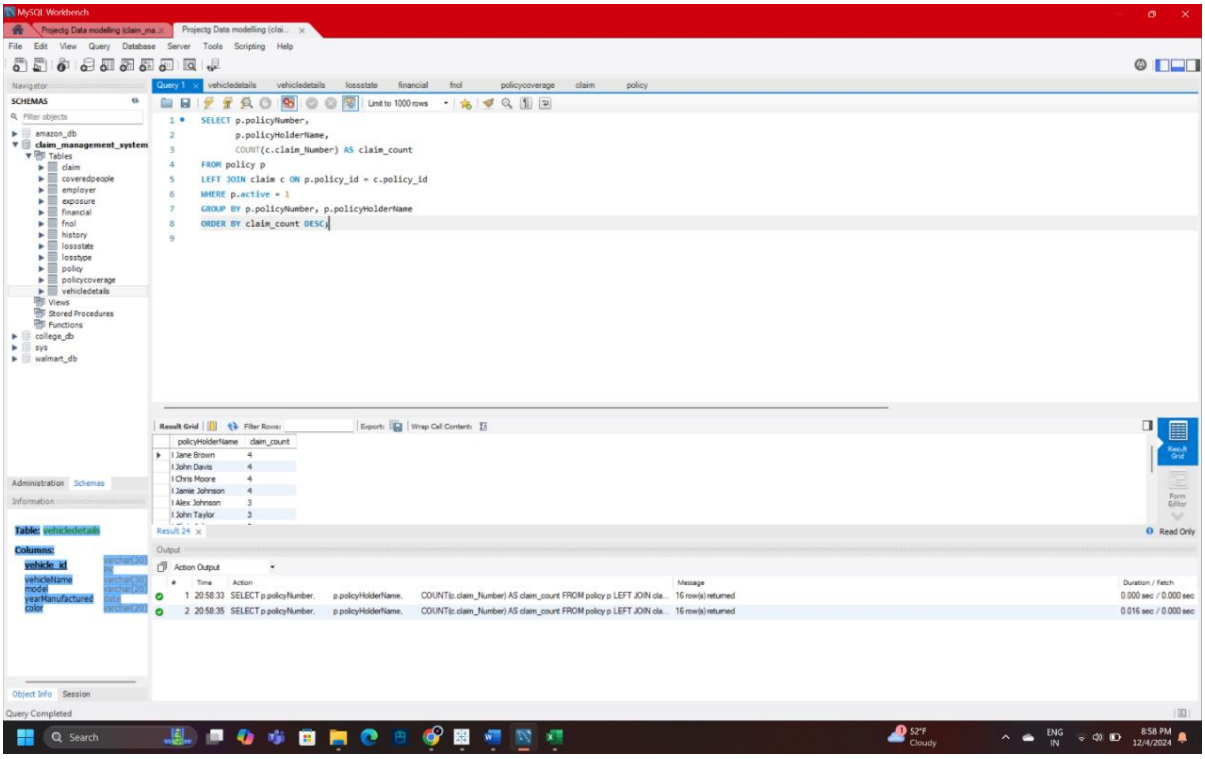
GROUP BY p.policyNumber, p.policyHolderName

ORDER BY claim_count DESC;

Explanation:

This query treats active policies i.e., policies whose status is shown by `p.active = 1` and counts for claims. Using the policy and claim tables it connects them using `policy_id` and select results grouped by policy details such as `policyNumber` and `policyHolderName` Most policies have been ordered depending on the number of claims a specific policy has received to display those with high activity in claims.

Result and Screenshot:



Analysis – 6:

This query describes the inactive policies that still have some claims yet to be resolved. Which helps the user to keep up with the stuff and solve the issues.

Query:

SELECT p.policy_id,

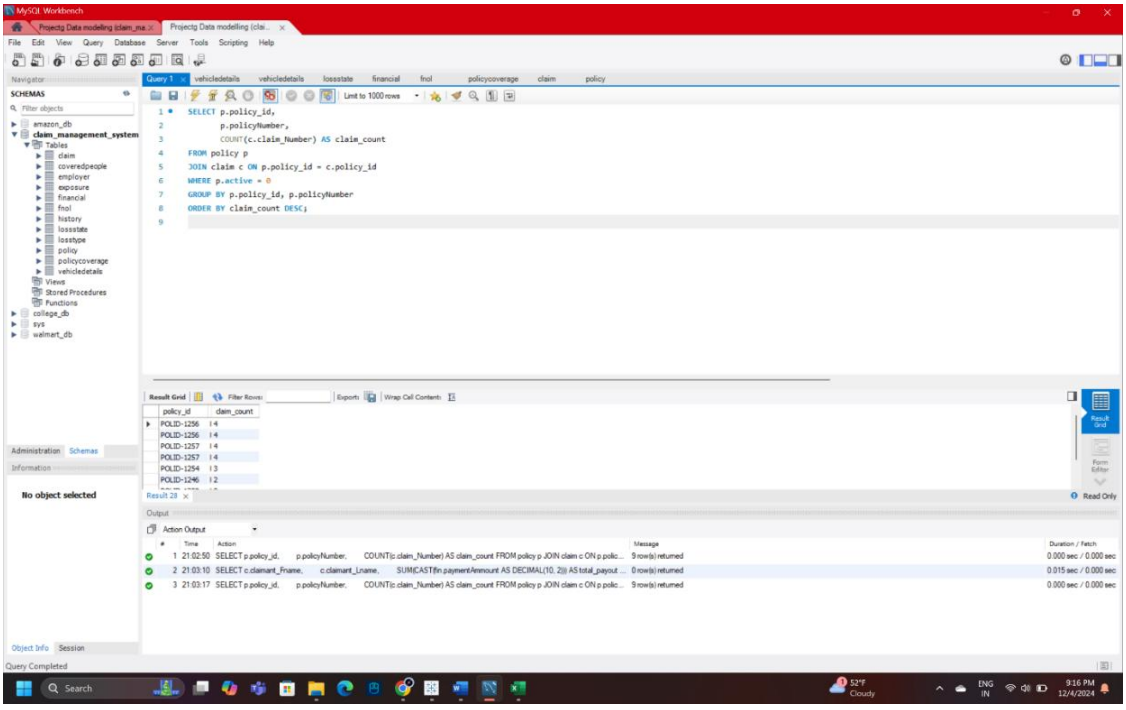
p.policyNumber,

```
    COUNT(c.claim_Number) AS claim_count
FROM policy p
JOIN claim c ON p.policy_id = c.policy_id
WHERE p.active = 0
GROUP BY p.policy_id, p.policyNumber
ORDER BY claim_count DESC;
```

Explanation:

This query involves policies that still have claims and are inactive (`p.active = 0`). It combines occurrences of `policy` and `claim` based on the policy ID, aggregates findings based on policy information (policy ID, policy number), quantifies accompanying claims, and ranks results based on the claim count, sorting for policies that experienced the greatest number of claims.

Result and Screenshot:



Analysis – 7:

Retrieving specific policies and claims by specific policyholders.

Query:

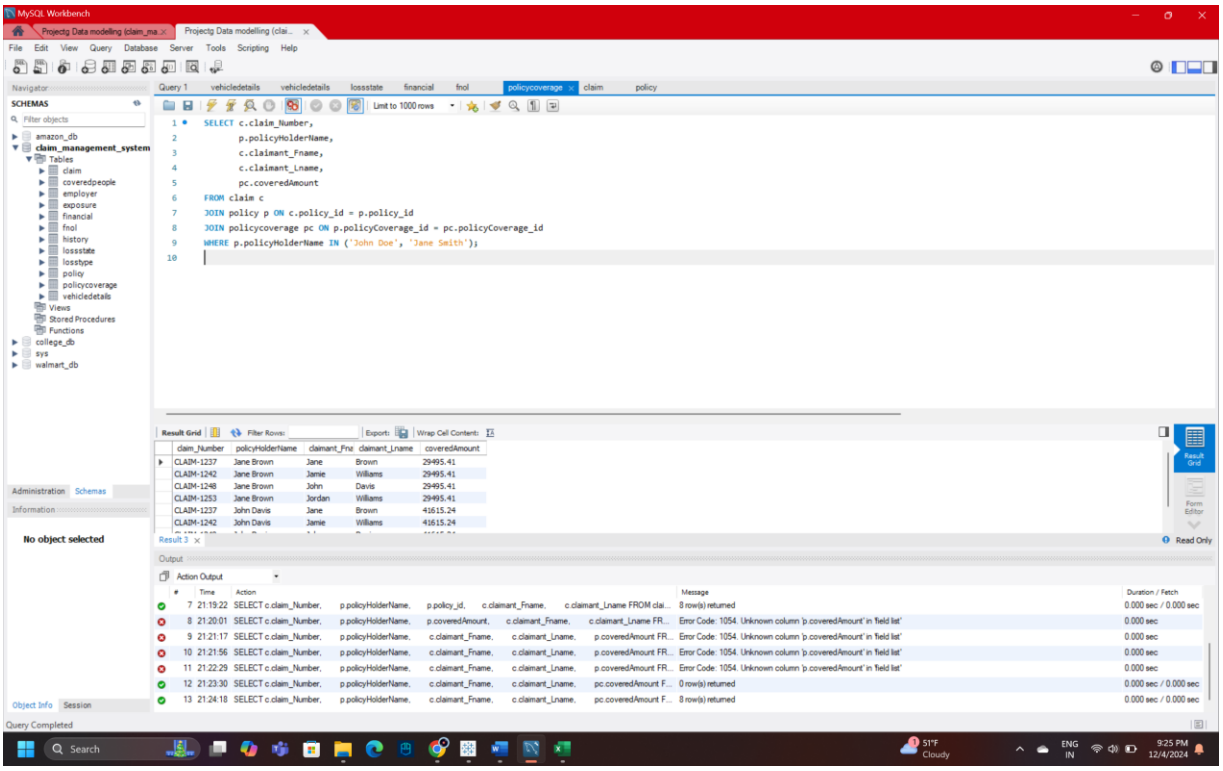
```
SELECT c.claim_Number,
```

```
    p.policyHolderName,
    c.claimant_Fname,
    c.claimant_Lname,
    pc.coveredAmount
FROM claim c
JOIN policy p ON c.policy_id = p.policy_id
JOIN policycoverage pc ON p.policyCoverage_id = pc.policyCoverage_id
WHERE p.policyHolderName IN ('John Doe', 'Jane Smith');
```

Explanation:

In this query, the user will get related claims that are associated with particular policyholders (policyHolderName). Combining policy and policycoverage tables by the policy_id and policyCoverage_id, the table is provided with the claim informations along with the claimant name and the coveredAmount.

Result and Screenshot:



Analysis – 8:

Find the most used payment type of the policyholders.

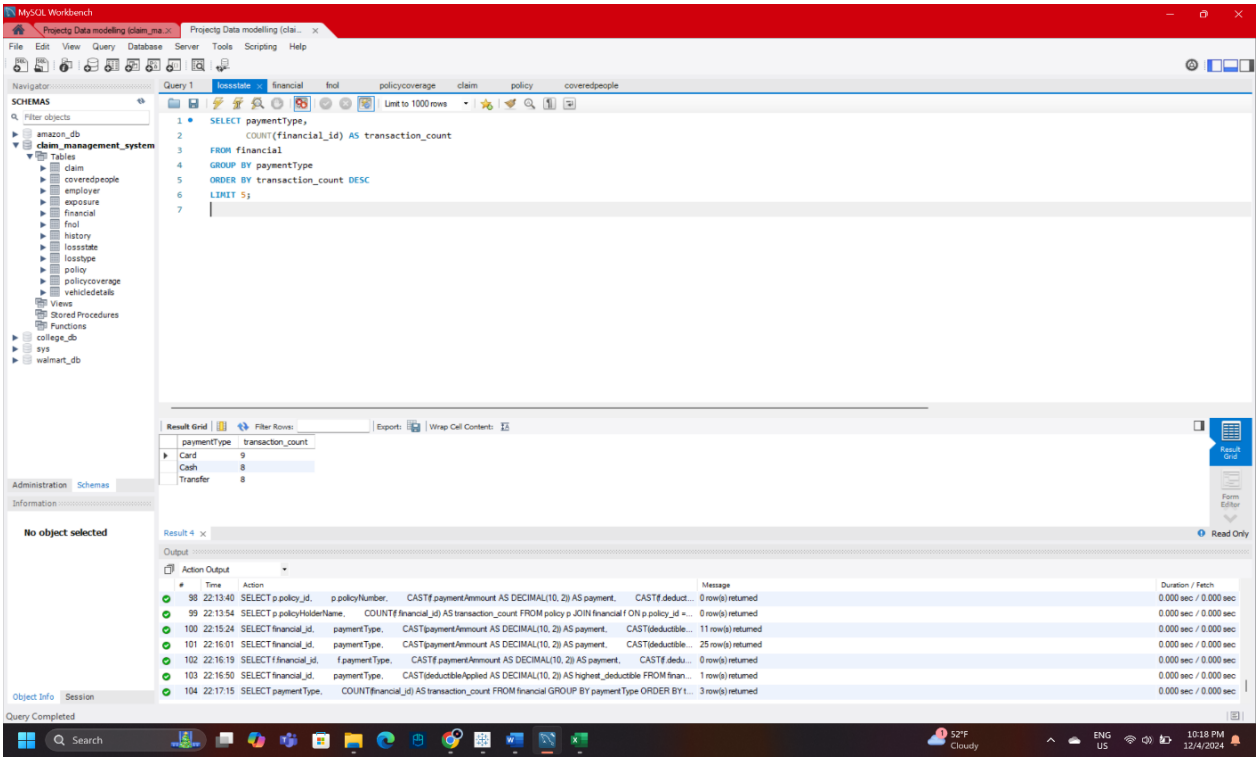
Query:

```
SELECT paymentType,
    COUNT(financial_id) AS transaction_count
FROM financial
GROUP BY paymentType
ORDER BY transaction_count DESC
LIMIT 5;
```

Explanation:

This query returns the most frequent payment types in the ‘financial’ table in the data base. It sorts data based on ‘paymentType’, calculates number of transactions as ‘financial_id’, sorts result in descending order of transaction count and limits it to 5 records.

Result and Screenshot:



Analysis-9:

Examining the trends between the payment status and the payment types.

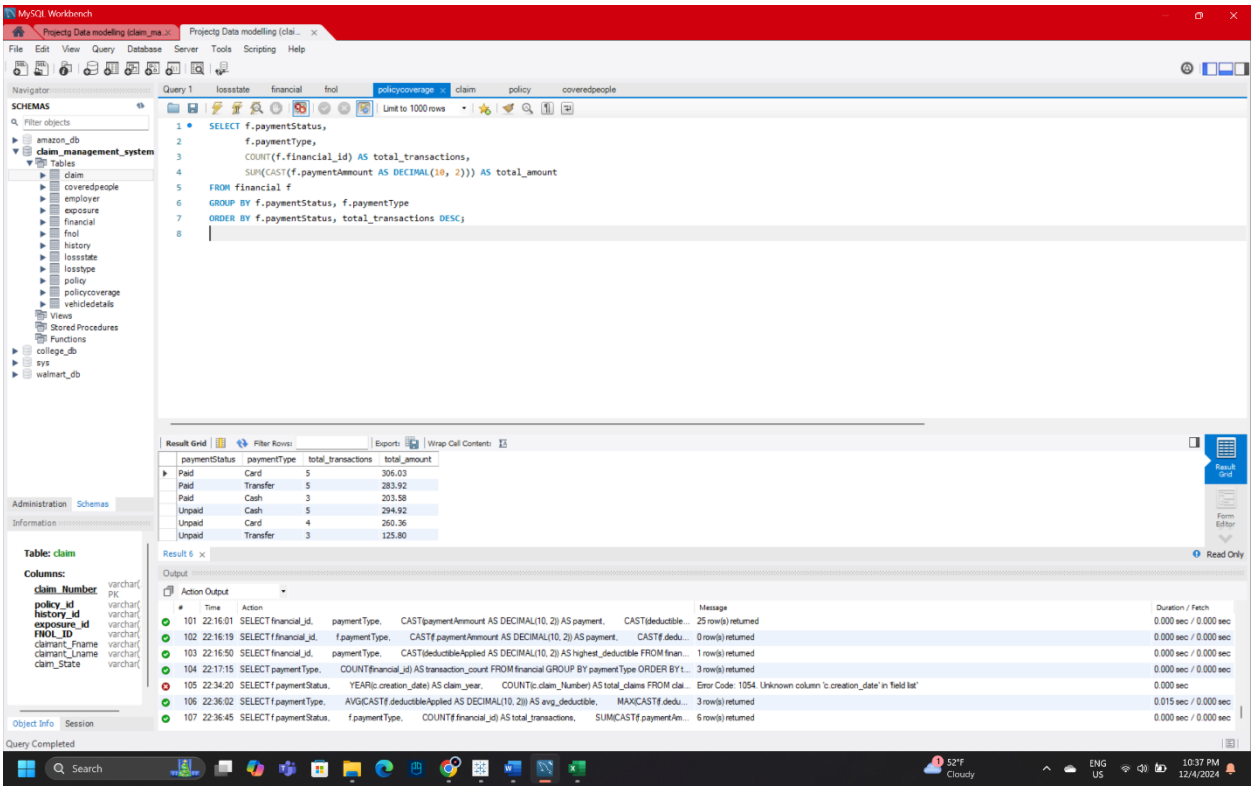
Query:

```
SELECT f.paymentStatus,
    f.paymentType,
    COUNT(f.financial_id) AS total_transactions,
    SUM(CAST(f.paymentAmmount AS DECIMAL(10, 2))) AS total_amount
FROM financial f
GROUP BY f.paymentStatus, f.paymentType
ORDER BY f.paymentStatus, total_transactions DESC;
```

Explanation:

The payment trends are evaluated by categorizing payment status and payment type from the `financial` table. It uses the concept of counts, sum, ordering of the result based on `paymentStatus` and transaction count to show trends on different locations based on Payment type and payment status. Responsive of the payment types and statuses discovered by trends in the transaction number and overall payment amounts.

Result and Screenshot:



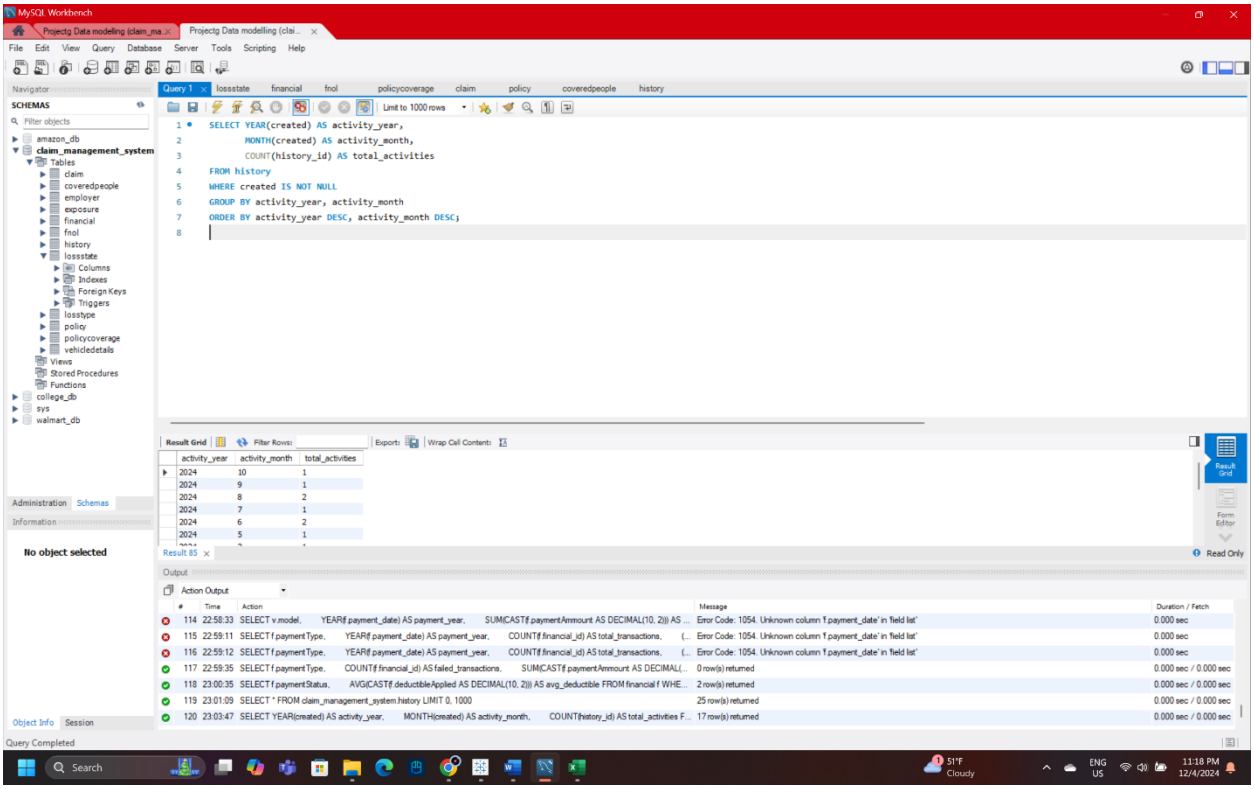
Analysis – 10:

To analyze and keep note of the actions done in months and years.

Query:

```
SELECT YEAR(created) AS activity_year,
    MONTH(created) AS activity_month,
    COUNT(history_id) AS total_activities
FROM history
WHERE created IS NOT NULL
GROUP BY activity_year, activity_month
ORDER BY activity_year DESC, activity_month DESC;
```

Explanation:

This query generates activity trends by the number of records developed yearly and monthly within the history table. They are arranged based on the year and month of action and filtered for the null creation date, arranged in descending order by year and then by month.

Result and Screenshot: